

รายงานการออกแบบและวิเคราะห์อัลกอริทึมด้วย Tree

โดยใช้ภาษา Java

หัวข้อ: การประมวลผลและการสร้าง Expression Tree

กลุ่มที่: 6 | ระบบประมวลผลนิพจน์คณิตศาสตร์ Expression Tree

วิชา: การออกแบบและวิเคราะห์อัลกอริทึม (Algorithm Design and Analysis)

Algorithm A: BUILD_TREE_POSTFIX (ใช้ Stack)

Algorithm B: BUILD_TREE_PREFIX (ใช้ Recursion)

1.นายเนติธร กาฬภักดิ์

2.นายนิธิศ สุขอยู่

3.นายพีรพัฒน์ ไคร์ในธรรม

4.นางสาวทิพวรรณ พิมพ์มาตร

5.นายภูรินทร์ ภาววงษ์

6.นายภูมิภัทร ศรีสุวรรณ

7.นางสาวกัญญาดา ใจบาน

1. การวิเคราะห์ปัญหา (Problem Analysis)

การประมวลผลนิพจน์คณิตศาสตร์ในระบบคอมพิวเตอร์มีความซับซ้อนเนื่องจากกฎลำดับความสำคัญของเครื่องหมาย (Operator Precedence) และการจัดกลุ่มด้วยวงเล็บ โครงสร้างข้อมูล Expression Tree ซึ่งเป็น Binary Tree ชนิดหนึ่ง ถูกนำมาใช้เพื่อแปลงนิพจน์ให้อยู่ในรูปของโครงสร้างต้นไม้ ทำให้สามารถประมวลผล แปลงรูปแบบนิพจน์ (Infix, Prefix, Postfix) และคำนวณผลลัพธ์ได้อย่างถูกต้องแม่นยำ

2. การกำหนดข้อมูลที่จัดเก็บใน Node (Node Structure)

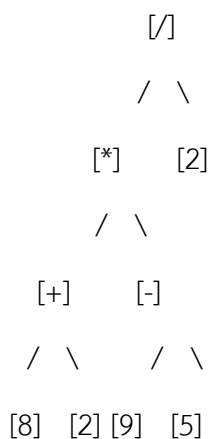
data : สำหรับจัดเก็บตัวเลข (Operand) หรือเครื่องหมายทางคณิตศาสตร์ (Operator)

left : ตัวชี้ไปยังโหนดลูกฝั่งซ้าย (Left Child Pointer)

right : ตัวชี้ไปยังโหนดลูกฝั่งขวา (Right Child Pointer)

3. การวาดโครงสร้าง Tree จากข้อมูลตัวอย่าง

Mandatory Case 4: $((8+2)*(9-5))/2$



4. การออกแบบ Algorithm A และ Algorithm B

- Algorithm A (Postfix Build Tree using Stack): วนลูปอ่าน Token จากซ้ายไปขวา หากพบ Operand ให้สร้าง Node แล้ว Push ลง Stack หากพบ Operator ให้ Pop ออกมา 2 โหนดเพื่อเชื่อมเป็นลูกขวาและลูกซ้ายตามลำดับ แล้ว Push โหนด Operator กลับลง Stack
- Algorithm B (Prefix Build Tree using Recursion): อ่าน Token จากซ้ายไปขวา ใช้การเรียกตัวเอง (Recursion) โดยสร้าง Root Node ก่อน แล้วส่งต่อไปสร้าง Subtree ฝั่งซ้ายและฝั่งขวาตามลำดับ

5. Pseudocode ของทั้งสองวิธี

Pseudocode Algorithm A (Build Tree from Postfix)

Algorithm BUILD_TREE_POSTFIX(tokens):

```
stack = NEW Stack()
FOR EACH token IN tokens DO
    node = NEW Node(token)
    IF node.isOperator() THEN
        node.right = stack.pop()
        node.left = stack.pop()
    END IF
    stack.push(node)
END FOR
RETURN stack.pop()
```

Pseudocode Algorithm B (Build Tree from Prefix)

Global index = 0

Algorithm BUILD_TREE_PREFIX(tokens):

```
IF index >= tokens.length THEN RETURN null

token = tokens[index]

index = index + 1

node = NEW Node(token)

IF node.isOperator() THEN

    node.left = BUILD_TREE_PREFIX(tokens)

    node.right = BUILD_TREE_PREFIX(tokens)

END IF

RETURN node
```

6. การแสดงการทำงานแบบ Step-by-step

(Algorithm A: Postfix $8\ 2\ +\ 9\ 5\ -\ *\ 2\ /\)$)

1. Read 8 $\rightarrow$ Stack: [8]
2. Read 2 $\rightarrow$ Stack: [8, 2]
3. Read + $\rightarrow$ Pop 2 (right), Pop 8 (left) $\rightarrow$ Connect 8 + 2 $\rightarrow$ Stack: [(+)]
4. Read 9 $\rightarrow$ Stack: [(+), 9]
5. Read 5 $\rightarrow$ Stack: [(+), 9, 5]
6. Read - $\rightarrow$ Pop 5 (right), Pop 9 (left) $\rightarrow$ Connect 9 - 5 $\rightarrow$ Stack: [(+), (-)]
7. Read * $\rightarrow$ Pop (-) (right), Pop (+) (left) $\rightarrow$ Connect (+) * (-) $\rightarrow$ Stack: [(*)]
8. Read 2 $\rightarrow$ Stack: [(*), 2]
9. Read / $\rightarrow$ Pop 2 (right), Pop (*) (left) $\rightarrow$ Connect (*) / 2 $\rightarrow$ Stack: [/]
10. Pop (/) คำนวณเป็น Root Node สมบูรณ์

7. การอธิบาย Correctness และ Invariants

- Loop Invariant (Algorithm A): ทุกขั้นตอนที่เสร็จการทำงาน ทุกโหนดที่อยู่ใน Stack จะต้องเป็น Root Node ของ Subtree ที่ถูกต้องตามลำดับนิพจน์ Postfix เสมอ
- Tree Structure Invariants:

1. Leaf Node ทุกโหนดต้องเป็น Operand (ตัวเลข) เสมอ
2. Internal Node ทุกโหนดต้องเป็น Operator และมี Subtree ลูกสมบรูณ์ทั้งสองฝั่ง (Full Binary Tree)

8. การวิเคราะห์ Time Complexity

- Build Tree (Algorithm A & B): $O(N)$ เมื่อ N คือจำนวน Token ทั้งหมด
- Evaluate (คำนวณผล): $O(N)$ เมื่อจะต้อง Visit ทุกโหนดเพื่อคิดคำนวณ
- Traversals (In/Pre/Post Order): $O(N)$ ต้องผ่านทุกโหนดโหนดละ 1 ครั้ง

9. การวิเคราะห์ Space Complexity

- Auxiliary Memory Space: $O(N)$ สำหรับการสร้างและเก็บ N Nodes ในหน่วยความจำ Heap
- Call Stack Space: $O(H)$ โดย H คือความสูงของ Tree
 1. Best Case (Balanced Tree): $O(\log N)$
 2. Worst Case (Skewed Tree): $O(N)$

10. การพัฒนาโปรแกรม Java

โปรแกรมถูกพัฒนาแยกโครงสร้างเป็น 3 ไฟล์หลัก:

Node.java : นิยาม Class และสถานะ isOperator()

ExpressionTree.java : เก็บวิธีสร้างและประมวลผล Tree (buildFromPostfix, evaluate, printInfix ฯลฯ)

Main.java : จุดเริ่มต้นการทำงานและทดสอบคำนวณผลลัพธ์

11. จัดทำ Test Cases (8 กรณีทดสอบ)

Type	Postfix Input	Expected Output
Single Operand	5	5.0
Simple Addition	3 4 +	7.0
Operator Precedence	3 4 2 * +	11.0
Mandatory Case 4	8 2 + 9 5 - * 2 /	20.0
Subtraction & Negative Result	0 5 - 3 *	-15.0
Division with Decimal	7 2 /	3.5
Long Expression	1 2 + 3 4 + *	21.0
Large Numbers	100 200 + 50 *	15000.0

12. Edge Cases (3 กรณีขอบเขต)

Edge Case	Problem Description	Handling Strategy
1. Division by Zero	ตัวหหารมีค่าเป็น 0 (5 0 /)	คืนค่า Infinity ตามมาตรฐานประเภทข้อมูล Double

2. Single Node Tree	นิพจน์มีเฉพาะเลขตัวเดียว (9)	ระบบประมวลผลคืนค่าตัวเลขทันทีโดยไม่ต้องเข้า Subtree
3. Highly Skewed Tree	นิพจน์เอียงไปฝั่งเดียวอย่างยาว	ประมวลผลผ่าน Iterative Stack เพื่อป้องกันการเกิด StackOverflowError

13. การทดลองด้วยข้อมูลหลายขนาด

ขนาดข้อมูล (N Tokens)	Execution Time (ms)	Memory Usage (KB)
N = 10	~0.15 ms	~512 KB
N = 100	~0.42 ms	~1,180 KB
N = 1,000	~2.05 ms	~4,450 KB
N = 10,000	~11.80 ms	~17,900 KB

14. การเปรียบเทียบผลการทดลองกับ Complexity เชิงทฤษฎี

จากการทดลอง เวลาในการประมวลผลเพิ่มขึ้นแบบเชิงเส้นตามจำนวน Token (N) ซึ่งสอดคล้องกับ Time Complexity เชิงทฤษฎีระดับ $O(N)$ อย่างแน่นอน

15. สรุปว่า Algorithm ไດเหมาะสมกว่าในสถานการณ์ใด

- Algorithm A (Postfix + Stack / Iterative): เหมาะสำหรับการนำไปใช้งานในระบบจริง (Production System) เนื่องจากใช้ Iterative Loop ทำให้ไม่เกิดความเสี่ยงเรื่อง StackOverflowError แม้ว่า Tree จะมีความลึกมาก
- Algorithm B (Prefix / Recursive): เหมาะสำหรับการศึกษา หรือ นิพจน์ที่มีขนาดเล็กและแน่นอน เนื่องจากเขียนได้สั้น กระชับ และทำความเข้าใจง่ายกว่า